CNP Rights

RBAC with Object-Level Overrides

This model uses Role-Based Access Control (RBAC) as the primary mechanism with two override layers for the cases where standard roles aren't enough.

Hierarchy

Every configuration value resolves through a three-layer hierarchy. The engine always returns the most specific value available, falling back up the chain if no override exists at a lower level.

The permission resolution algorithm always returns the most specific applicable permission. Layer 3 overrides Layer 2 overrides Layer 1. An explicit Revoke at any layer beats a Grant at a lower layer. Cannot work around a revocation by granting at a lower layer.

Layer 1 — Role Permissions

The base set of permissions every role has by default across all states. StateReviewer always includes SNP.Applications.Approve.

Lives in cnp_global
Example: LateClaimCutoffDays = 6

if tenant override exists, it moves to layer 2→

Layer 2 — Tenant Role Overrides

State-specific additions or removals from the base role. Utah grants StateReviewer access to DirectVer. Maine removes the auto-approve permission

Lives in cnp_ops

if user level override exists, it moves to layer 3 →

Layer 3 — User Permission Overrides

Individual user grants or revocations. Temporary access for a contractor. Restricted access for a user under investigation.

Requires justification + expiry date
Lives in cnp_ops.

⚠ **IsFederal blocks revocation at all layers**

Certain permissions are federally required for certain roles. An FNS Auditor must be able to read any compliance record they're auditing; a State Agency Admin must be able to approve applications. Permissions marked IsFederal=true on the RolePermissions row cannot be revoked by tenant overrides or user overrides. The engine rejects the write attempt, same as the workflow guardrail mechanism.

Tables

PermissionCatalog (cnp_global)

Column	Type	Notes
PermissionId	PK	
PermissionCode	varchar	Dotted key: `SNP.Applications.Approve`, `CACFP.Claims.Submit`, `Accounting.Reports.Export`
Domain	varchar	SNP, CACFP, SFSP, Accounting, Configuration, DC, Compliance, Global
SubArea	varchar	Applications, Claims, Compliance, Reports, Configuration, Security, Search, Dashboard
Action	varchar	View, Create, Edit, Submit, Approve, Lock, Export, Configure, Manage, Impersonate
Description	text	Plain-English description of what this permission allows
IsActive	bool	Inactive permissions still exist for audit purposes but grant nothing

RoleDefinitions (cnp_global)

Column	Type	Notes
RoleId	PK	
RoleCode	varchar	e.g. `StateAgencyAdmin`, `Reviewer`, `Caseworker`, `SponsorDirector`
RoleName	varchar	Display name
Description	text	

ScopeConstraint	varchar	`CrossTenant` (LINQ Admin, FNS Auditor), `SingleTenant` (most roles), `SponsorOnly` (sponsor staff)

RolePermissions (cnp_global)

Column	Type	Notes
RolePermId	PK	
RoleId	FK → RoleDefinitions	
PermissionId	FK → PermissionCatalog	
IsFederal	bool	True = cannot be revoked by any tenant or user override
IsDefault	bool	True = all tenants get this role+permission combo automatically. False = tenant must explicitly grant.

TenantRoleDefinitions (cnp_ops)

Column	Type	Notes
TenantRoleId	PK	
TenantCode	varchar	The tenant this role belongs to
RoleCode	varchar	e.g. `SNP-Only-Caseworker`, `CACFP-Claims-View`
RoleName	varchar	Display name
Description	text	
ScopeConstraint	varchar	`SingleTenant` or `SponsorOnly` only — CrossTenant is

		reserved for system roles
CreatedBy	FK → WebUsers	Onshore config team member
CreatedAt	timestamp	
IsActive	bool	Soft delete — deactivated roles retained for audit

TenantRolePermissions (cnp_ops)

Column	Type	Notes
TenantRolePermId	PK	
TenantRoleId	FK → TenantRoleDefinitions	
PermissionId	FK → PermissionCatalog	Still references the global catalog
GrantedBy	FK → WebUsers	
GrantedAt	timestamp	

TenantRoleOverrides (cnp_ops)

Column	Type	Notes
OverrideId	PK	
RoleId	FK → RoleDefinitions	
PermissionId	FK → PermissionCatalog	
TenantCode	varchar	
OverrideType	varchar	`Grant` (add a permission to a role for this tenant) or `Revoke` (remove one)

EnabledBy	FK → WebUsers	Onshore config team member
EnabledAt	timestamp	
Justification	text	Required. Written reason for the override.
Engine rejects writes where RolePermissions.IsFederal = true for the target RoleId + PermissionId.		

UserRoleAssignments (cnp_ops)

Column	Type	Notes
AssignmentId	PK	
UserId	FK → WebUsers	
RoleId	FK → RoleDefinitions	
TenantRoleId	FK null → TenantRoleDefinitions	Populated when this is a tenant-defined role assignment. Exactly one of RoleId or TenantRoleId must be non-null.
TenantCode	varchar	The state this role applies to. A user can be Reviewer for Utah and Caseworker for Maine simultaneously.
ScopeType	varchar	`AllSponsors`, `SpecificSponsor`, `OwnDataOnly` — see Section 8
SponsorId	FK null	Populated only when ScopeType = SpecificSponsor
AssignedBy	FK → WebUsers	
AssignedAt	timestamp	
ExpiresAt	timestamp null	Null = permanent. Set for temporary access grants (contractors, auditors with time-limited access).
DomainScope	varchar null	When set, narrows this role assignment to a specific program domain (e.g. `SNP`, `CACFP`, `SFSP`, `FFVP`, `FDP`, `Accounting`, `Compliance`). Null = all domains the role's permissions cover.

		Mirrors the SponsorId pattern: a single column that eliminates the need for per-program role variants.

⚠ **Constraint** Exactly one of RoleId or TenantRoleId must be populated per row. Engine rejects writes where both are null or both are non-null.

UserPermissionOverrides (cnp_ops)

Column	Type	Notes
OverrideId	PK	
UserId	FK → WebUsers	
PermissionId	FK → PermissionCatalog	
TenantCode	varchar	
OverrideType	varchar	`Grant` or `Revoke`
Reason	text	Required. Explains why this individual user has an exception.
GrantedBy	FK → WebUsers	Must have Manage permission on the Security domain
GrantedAt	timestamp	
ExpiresAt	timestamp null	Strongly recommended. Individual overrides without expiry dates accumulate into permission debt.

Roles Catalog

StateAgencyAdmin

- Full access to all modules for their state
- Can assign roles to other state users

Reviewer

- View + approve applications and claims
- Initiate and conduct compliance reviews
- Calculate fiscal actions

Caseworker

- Process applications and claims
- View compliance records
- Run DC matching

- Can create tenant role overrides
- Cannot access another state's data
- ScopeConstraint: SingleTenant

- Cannot configure system settings
- ScopeConstraint: SingleTenant

- Cannot approve or lock records
- ScopeConstraint: SingleTenant

SponsorDirector

- View and submit sponsor's own applications and claims
- View sponsor's compliance history
- Manage sponsor user accounts
- ScopeConstraint: SponsorOnly (own sponsor)

SponsorStaff

- Enter meal counts, submit claims
- View own sponsor's application status
- No access to financial data
- ScopeConstraint: SponsorOnly (own sponsor)

Finance

- Full access to Accounting domain
- FNS report generation and export
- Grant balance management
-
- No access to program operational data
- ScopeConstraint: SingleTenant

LINQConfigAdmin

- Manage CnpDefaults and workflow overrides
- Manage DC feed configurations
- Can operate across all tenants
- No access to program operational data
- ScopeConstraint: CrossTenant

FNSAuditor

- Read-only access to all records in assigned state(s)
- Access to full audit trail and compliance history
- Cannot modify any record
- Time-limited access (ExpiresAt always required)
- ScopeConstraint: CrossTenant (read-only)

LINQPlatformAdmin

- Full platform access including cross-tenant queries
- Can view production data for support purposes
- All actions logged with CrossTenantFlag=true
- LINQ internal only — never granted to state users
- ScopeConstraint: CrossTenant

Permission Catalog Structure

Actions vocabulary

Action	What it grants	Examples
`View`	Read records. Cannot modify anything.	SNP.Claims.View, CACFP.Applications.View
`Create`	Create new records	SNP.Applications.Create, SFSP.Sites.Create
`Edit`	Modify draft/in-progress records	SNP.Claims.Edit, Accounting.Disbursements.Edit
`Submit`	Trigger Submit workflow transition	SNP.Claims.Submit, CACFP.Applications.Submit
`Approve`	Trigger Approve workflow transition	SNP.Applications.Approve, SNP.Claims.Approve
`Lock`	Trigger Lock/finalize transition	SNP.Claims.Lock, Accounting.Disbursements.Lock
`Export`	Download data or generate reports	SNP.Reports.Export, Accounting.FNSReports.Export
`Configure`	Modify configuration settings	Configuration.Maintenance.Configure
`Manage`	Administer users and roles	Security.Users.Manage, Security.Roles.Manage

Reports are governed by domain access. A user with `SNP.Claims.View` can view SNP claim reports. A user with `SNP.Claims.Export` can export them. Report-specific gates (e.g., FNS-10 is Finance-only) are handled by the `Accounting.FNSReports.Export` permission. This collapses the existing report permission sprawl into the domain model.

The PermissionCatalog has approximately 180 entries: 8 domains × ~4 sub-areas × ~5 actions, plus some domain-specific extras. The state-specific variation is handled by TenantRoleOverrides, not by individual permission entries.

How Permissions Resolve at Runtime

The service is cached per-request so it doesn't re-query the database on every permission check within a single request. The cache is invalidated when a UserRoleAssignment or TenantRoleOverride is written, and at midnight UTC for any expiring assignments.

```
// ICnpPermissionService — the only interface application code uses for authorization
public interface ICnpPermissionService {
  bool HasPermission(string userId, string tenantCode, string permissionCode);
  IEnumerable<string> GetPermissions(string userId, string tenantCode);
  bool HasRole(string userId, string tenantCode, string roleCode);
  bool HasSponsorAccess(string userId, string tenantCode, int sponsorId);
}

// Resolution algorithm (simplified)
bool HasPermission(userId, tenantCode, permissionCode) {

  // 1. Check user-level override first (highest priority)
  var userOverride = UserPermissionOverrides
    .Where(o => o.UserId == userId && o.TenantCode == tenantCode
             && o.PermissionCode == permissionCode
             && (o.ExpiresAt == null || o.ExpiresAt > NOW()))
    .FirstOrDefault();
  if (userOverride?.OverrideType == "Revoke") return false;
  if (userOverride?.OverrideType == "Grant") return true;

  // 2. Extract domain from permission code (e.g. "SNP" from "SNP.Claims.View")
  //    and get user's role assignments for this tenant, filtered by DomainScope when set
  var domain = permissionCode.Split('.')[0];

  var assignments = UserRoleAssignments
    .Where(r => r.UserId == userId && r.TenantCode == tenantCode
             && (r.ExpiresAt == null || r.ExpiresAt > NOW())
             && (r.DomainScope == null || r.DomainScope == domain));

  var systemRoleIds = assignments.Where(a => a.RoleId != null).Select(a => a.RoleId);
  var tenantRoleIds = assignments.Where(a => a.TenantRoleId != null).Select(a =>
a.TenantRoleId);

  // 3. Check system roles (cnp_global) — TenantRoleOverride and IsFederal logic applies
  foreach (var roleId in systemRoleIds) {
    var tenantOverride = TenantRoleOverrides
      .Where(o => o.RoleId == roleId && o.TenantCode == tenantCode
               && o.PermissionCode == permissionCode)
      .FirstOrDefault();
    if (tenantOverride?.OverrideType == "Revoke") continue;
    if (tenantOverride?.OverrideType == "Grant") return true;

    // Fall back to base role permission
    if (RolePermissions.Any(rp => rp.RoleId == roleId && rp.PermissionCode ==
permissionCode))
      return true;
  }

  // 4. Check tenant-defined roles (cnp_ops) — permissions are direct, no overrides apply
  foreach (var tenantRoleId in tenantRoleIds) {
    if (TenantRolePermissions.Any(rp => rp.TenantRoleId == tenantRoleId
                                     && rp.PermissionCode == permissionCode))
      return true;
  }

  // 5. No grant found at any layer
  return false;
}
```

Guardrails:

- TenantRoleDefinitions.ScopeConstraint cannot be `CrossTenant`

- TenantRolePermissions can only reference permissions the tenant's `StateAgencyAdmin` system role already has — a tenant-defined role cannot exceed the ceiling of the highest system role

The OIDC Connection

The OIDC broker handles **authentication** (who you are). The ICnpPermissionService handles **authorization** (what you can do). These are deliberately separated.

The OIDC token includes the user's role assignments and tenant scopes so the application can bootstrap the permission service without a database roundtrip on every request. It does not include the full permission list.

JWT claims payload (relevant fields)

```
{
  "sub": "usr-14422",
  "email": "reviewer@mn.gov",
  "cnp_roles": [
    { "role": "Reviewer", "tenant": "mn", "scope": "AllSponsors" },
    { "role": "Caseworker", "tenant": "ia", "scope": "AllSponsors", "domain": "SNP" }
  ],
  "cnp_tenants": ["mn","ia"],          // tenants this user is active in
  "cnp_cross_tenant": false,    // true only for LINQPlatformAdmin and FNSAuditor
  "exp": 1781300000
}
```

The broker validates, the service resolves The OIDC broker validates that the token is authentic (signature, expiry, issuer). The ICnpPermissionService resolves what permissions those roles carry for the requested tenant, applying all overrides. Application code calls ICnpPermissionService. It never parses JWT claims directly for authorization decisions.

The optional `domain` field indicates a domain-scoped assignment. When present, the permission service only applies that role's permissions when evaluating permissions in the matching domain. Omitting `domain` is equivalent to null — the role applies across all domains.

Sponsor-Scoped Access

The most nuanced part of CNP rights is sponsor-scoped access. A school district director should be able to view and submit their own district's claims, but not see other districts' claims. A state agency reviewer should be able to see all sponsors in their state. A LINQ config admin should be able to see configuration across all states but not operational sponsor data.

The `ScopeType` on UserRoleAssignments handles this without requiring object-level ACLs for every sponsor.

ScopeType	What the user can	Who uses it

`AllSponsors`	All sponsors within their assigned tenant(s)	StateAgencyAdmin, Reviewer, Caseworker, Finance
`SpecificSponsor`	Only the sponsor identified by SponsorId FK	SponsorDirector, SponsorStaff — scoped to their own organization
`OwnDataOnly`	Only records they personally created or are assigned to	Temporary reviewers, interns — limited view during training

The ICnpPermissionService's `HasSponsorAccess(userId, tenantCode, sponsorId)` method encapsulates this check. Application code calls it before returning any sponsor-scoped data. The query filter is injected automatically at the repository layer for all sponsor-scoped entities.

A sponsor user's access to their own data is enforced by individual permission rows per object. In the new model, ScopeType=SpecificSponsor on their role assignment is a single row that enforces the boundary for every entity type in the system so there's no per-object permission management required.

Domain-Scoped Access

`DomainScope` on UserRoleAssignments is the program-dimension parallel to `SponsorId`. Where sponsor scoping answers "which organizations can this user see?", domain scoping answers "which programs can this user operate in?" States with separate staff for different nutrition programs assign Caseworker with DomainScope = 'SNP' to one group and DomainScope = 'CACFP' to another. A second assignment can be added independently if responsibilities expand.

IsFederal Guardrails in the Rights Model

The same IsFederal guardrail pattern used in the workflow engine and CnpDefaults applies here. Certain role-permission combinations are federally required and cannot be revoked by any tenant or user override.

Role	Permission	Why IsFederal
FNSAuditor	Any domain.View	Federal regulations require USDA auditors to have read access to any record they are auditing. A state cannot restrict this.

StateAgencyAdmin	SNP.Applications.Approve	USDA requires a state agency official to approve applications. The state cannot remove this from the Admin role.
Reviewer	Compliance.Reviews.Create	USDA requires state agencies to conduct compliance reviews. The state cannot prevent reviewers from initiating them.
LINQPlatformAdmin	Security.Users.Manage	Platform administration requires user management capability. This cannot be revoked.

⚠ **IsFederal =! "the user has this permission forever"**

IsFederal on a RolePermission means the permission cannot be *removed from the role* via an override. A user can still lose the permission by having their role assignment revoked entirely.

The distinction: IsFederal protects the role definition; it doesn't protect the user assignment. A state agency can remove someone from the Reviewer role. They cannot make the Reviewer role unable to create compliance reviews.

Migration from Existing CnpObjects

Migrating from 5,082 individual ACL rows to the new RBAC model is a data exercise, not a code exercise. The architect's domain document gives us the structure we need — the 12 domains and 67 sub-areas map directly to the PermissionCatalog's Domain + SubArea columns.

Migration sequence

Step	What happens	Who
1. Catalog extraction	AI analyzes existing CnpObjects rows and produces a structured inventory of unique permission types, grouped by domain and sub-area. Output: draft PermissionCatalog with ~180 rows.	AI-assisted, onshore validation
2. Role mapping	Each existing user's permission set is clustered against the new role definitions. AI identifies which role best represents each user's permissions. Output: draft UserRoleAssignments.	AI-assisted, onshore review

3. Exception identification	Users whose permissions don't cleanly map to a single role get flagged. These become TenantRoleOverrides or UserPermissionOverrides.	AI-assisted, onshore sign-off
4. Tenant variation analysis	Permissions that differ across states for the same role type become TenantRoleOverrides. Same pattern as DIS state configuration profiling.	Onshore config team
5. Parallel validation	New RBAC model runs in shadow mode alongside existing CnpObjects for one SY cycle. ICnpPermissionService results compared against existing access grants. Discrepancies investigated and resolved.	Offshore QA + onshore validation
6. Cutover	CnpObjects deprecated, RBAC model becomes authoritative. Old ACL rows retained read-only for audit purposes.	Offshore deploy + onshore confirm

Phase 1 scope: SECURITY domain only

The Phase 1 Security & Configuration module means migrating the SECURITY domain first . This is the smallest domain and the natural starting point. Once the RBAC model is proven for SECURITY, CONFIGURATION follows, and then the program domains in Phase 3 order (SNP → CACFP → SFSP).

DIS/CTS convergence through TenantRoleOverrides DIS states have different access patterns than CTS states. In the new model these differences become TenantRoleOverrides. Same engine, same roles, different per-tenant configuration.